Comp 333
Group project
Phase Final
Deadline December 3rd

Team members: Joel Mwamba, Karim Mikhaeil, Omar Sarayrah, Kabeya Ngoyi, Mohammed Al-Khateeb

Flight Delay Prediction and Analysis Across U.S. Airports

A- Data Integration & cleaning (Omar & Karim)....6
1. Introduction....6
2. Data Sources (In Depth)....7
2.1 GitHub Dataset (Monthly CSVs)....7
2.2 RowZero Dataset....7
2.3 Kaggle Delay Cause Dataset....7
2.4 Mendeley Dataset....8
3. Data Quality Assessment....8
3.1 Missing Values....8
3.2 Format Issues....8
4. Data Integration Approach (Detailed)....9
4.1 Directory Automation....9
4.2 Schema Standardization....9
4.3 Merging Airline Names....9
4.4 Merging Airport Metadata....9
4.5 Selection of Essential Columns....10
5. Feature Engineering (Advanced)....10
5.1 TOTAL_DELAY Construction....10
5.2 Why TOTAL_DELAY Matters....10
5.3 Additional Features Possible (Future Work)....11
6. Exploratory Insights....11
6.1 Top 5 Airlines....11
6.2 Top 5 Airports....11
6.3 Basic Delay Trends (Initial Observations)....12
7. Export Strategy....12
7.1 Problem....12
7.2 Solution....12
7.3 Result....12
8. Export Strategy (Expanded and In-Depth Analysis)....13
8.1 Rationale Behind the Export Design....13
1. Noise and Redundancy....13
2. Schema Fragmentation Across Sources....13
3. Analytical Relevance....14
8.2 Final Dataset Schema Design....14
Core flight information....14
Operational timing fields....14
Delay fields....14
Engineered feature....14
Flight status....15
8.3 Export Logic and Data Preparation....15
8.3.1 Dropping irrelevant columns....15

8.3.2 Resolving missing values....15
8.3.3 Standardizing datatypes.... 15
8.4 Justification for CSV as the Export Format....15
1. Universally supported.... 16
2. Human-readable structure.... 16
3. Long-term archival.... 16
B- Data Cleaning & Transformation (Joel)....17
1. Data Examination....17
2. Stability and Backup.... 18
4. Target Variable Encoding.... 18
5. Train/Test Split.... 19
6. Categorical Encoding....19
7. Attribute Reduction.... 19
8. Final Dataset Preparation.... 21
C- Data Visualization & ETL (Kabeya)....22
D- Modeling & Deployment (Mohammed)....25
1. Model Choice: Why I Used a Decision Tree.... 25
2. Before Cleaning / Transformation (Baseline Model).... 25
3. After Cleaning + Transformation....26
4. What the F1 Score Represents....27
5. Deployment: Predicting a New Flight....27
Example 1 — Predicted as Delayed.... 28
Example 2 Predicted as On Time.... 28
6. How the Model Behaves with Incorrect or Unseen Inputs(Optional Considerations).. 29
7. Analysis & Findings....30
Final Conclusion & Outcomes of the Project....31
1. Data Integration & Initial Cleaning.... 31
2. Data Cleaning & Transformation....31
3. Data Visualization & ETL.... 31
4. Modeling & Deployment....32
Baseline Model (Before Cleaning).... 32
Final Model (After Cleaning & Transformation)....32
5. Main Outcomes & What We Learned.... 32
Clean Data Makes the Biggest Difference....32
Delay Causes Play a Strong Role....33
Some Airports and Airlines Are More Delay-Prone....33
A Simple Decision Tree Was Enough....33
The Pipeline Worked End-to-End.... 33
Final Statement....34
Use Of AI.... 34

Project Definition

Our project focuses on predicting flight delays using historical U.S. flight-performance data. The objective is to determine whether a flight or an airport in a given month is likely to be delayed or on time. This is a classification task, where the model outputs one of two categories: Delayed (1) or On Time (0).

The dataset includes several features that affect flight operations, such as airport, airline, time of day, month, and the main causes of delay (weather, air-traffic system delays, carrier delays, and late-aircraft delays). These factors represent both operational and environmental conditions that can contribute to disruptions in the schedule.

The goal of the project is to organize, clean, and transform this data so we can build a model that identifies patterns linked to delays. By doing this, we can better understand which factors play the biggest role in delay outcomes and create a predictive system that estimates the likelihood of a flight being delayed based on its characteristics.

A- Data Integration & cleaning (Omar & Karim)

1. Introduction

Flight delays are a pervasive operational challenge for airlines, airports, and passengers across the United States. Understanding the factors that contribute to delays requires aggregating information from multiple data sources, including flight schedules, actual performance times, delay causes, weather data, and carrier/airport metadata.

However, these data sources are fragmented, inconsistent, and exist in different formats, making integration a complex but necessary step for accurate analysis and predictive modeling.

This project consolidates four major datasets, including millions of records, into a unified relational dataset designed for:

- Predictive modeling (delay classification and regression)
- Airline and airport performance analysis
- Exploratory data analysis (EDA)
- Operational decision support

The final dataset is extracted into a clean CSV file under 800MB, making it usable on typical hardware while preserving analytical value.

---

2. Data Sources (In Depth)

2.1 GitHub Dataset (Monthly CSVs)

This is the core dataset, containing detailed flight-level information for all U.S. domestic flights in a full year.

- Size: ~6.8M rows
- Columns: 31
- Key Information:
    - Scheduled & actual times
    - Departure & arrival delays
    - Taxi-out/in times
    - Delay causes (carrier, weather, NAS, security, late aircraft)
    - Cancellation information

This dataset provides the foundation for the analysis: each row represents one flight event.

Key issue:
More than 400,000 missing values exist for the delay cause columns because these fields are filled only when the airline reports specific causes.

2.2 RowZero Dataset

A large dataset providing:

- Airline code, flight date
- Delay values
- Cancellation status

Although smaller than the GitHub dataset in complexity, it helped validate column consistency and missingness patterns.

2.3 Kaggle Delay Cause Dataset

- Monthly aggregated delay statistics
- Grouped by airline and airport
- Includes cause-level delay totals (weather, NAS, security...)

Used to understand macro-level delay patterns and enrich flights with high-level delay trends.

2.4 Mendeley Dataset

This dataset combines:

- Flight details
- Operational variables
- Weather conditions (temperature, precipitation, wind)

While not directly merged (due to size constraints), it was essential for understanding the relationship between weather and delays.

---

3. Data Quality Assessment

3.1 Missing Values

Across all datasets, the following fields had significant missingness:

Column	Issue
DEP_TIME	Flights that never departed (cancellations)
DEP_DELAY	Missing for cancelled/diverted flights
ARR_TIME	Missing for cancellations
CARRIER_DELAY, WEATHER_DELAY, etc.	~400k missing per month
CANCELLATION_CODE	>90% missing

3.2 Format Issues

- Date formats differed across datasets
- Carrier codes inconsistent (OP_UNIQUE_CARRIER vs uniquecarrier)
- Airport identifiers in different cases or formats
- Time values stored as integers instead of timestamps

These issues required standardization.

---

4. Data Integration Approach (Detailed)

4.1 Directory Automation

All monthly CSV files were loaded automatically via Python by identifying them inside:

/github/monthly/

This avoided manual loading and ensured reproducibility.

4.2 Schema Standardization

Key steps:

1. Convert all airline identifiers to CARRIER_CODE
2. Convert airport codes to uppercase IATA codes
3. Ensure date columns follow ISO format
4. Rename columns to a unified schema

This step is crucial because inconsistent field names prevent merging datasets reliably.

4.3 Merging Airline Names

Airline codes (e.g., UA, DL, AA) were enriched with full airline names using Airline.csv.

This enhances readability and supports high-level airline comparisons.

4.4 Merging Airport Metadata

Airport codes were enriched with location data:

- Airport name
- City
- State
- Latitude
- Longitude

This metadata can support analyses on:

- Geographic delay patterns
- Weather severity by state
- Congestion clustering

4.5 Selection of Essential Columns

Given size constraints, not all fields were kept. Instead, you selected columns strategically to preserve:

- Delay behavior
- Operational performance
- Route analysis
- Time-based features
- Airline + airport identity

This selection ensured the final CSV remained under 800MB, which is feasible for local processing.

---

5. Feature Engineering (Advanced)

5.1 TOTAL_DELAY Construction

Delay is not a single value; it is composed of:

- Carrier delay (maintenance/crew issues)
- Weather delay
- NAS delay (air traffic control)
- Security delay
- Late aircraft delay
- Arrival delay
- Departure delay

By summing all available sources:

$$TOTAL_DELAY = \Sigma(delay_columns)$$

This creates a unified metric that reflects the complete operational impact of the flight.

5.2 Why TOTAL_DELAY Matters

- Supports supervised ML (regression/classification)
- Enables ranking of worst-performing airlines/airports
- Allows correlation analysis (weather impact, ATC impact)
- Identifies systemic causes

5.3 Additional Features Possible (Future Work)

You can still derive:

- Delay Binary Label
  DELAYED = 1 if ARR_DELAY > 15
- Hour & Day Features
  - Scheduled departure hour
  - Day of week
  - Month
  - Holiday indicators
- Airport Congestion Index
  - Flights per hour at each airport
- Weather Severity Score
  - Derived from Mendeley dataset

These improve predictive performance significantly.

---

6. Exploratory Insights

6.1 Top 5 Airlines

Using flight count:

- Helps identify major carriers dominating U.S. skies
- High traffic carriers often have larger operational challenges
- Visualization provided clear differences between carrier scales

---

6.2 Top 5 Airports

Using origin airport frequency:

- Identifies major hubs (ATL, ORD, DFW, LAX, DEN)
- These airports often experience:
  - Congestion delays
  - Weather bottlenecks
  - Higher operational complexity

Understanding top airports helps interpret delay trends.

6.3 Basic Delay Trends (Initial Observations)

Even without full modeling, preliminary analysis suggests:

- Late Aircraft Delay is one of the biggest contributors
- NAS (Air Traffic Control) delays spike during busy travel periods
- Weather Delays are more seasonal (winter storms, summer thunderstorms)
- Cancellations correlate strongly with:
    - Major weather events
    - Crew shortages
    - Severe ATC restrictions

---

7. Export Strategy

7.1 Problem

A full dataset with all columns exceeded 5–10 GB, which:

- Cannot fit in memory
- Cannot be exported
- Causes OSError: No space left

7.2 Solution

You used a column-reduced integrated dataset storing only essential columns.

7.3 Result

The final CSV:

- Under 800MB
- Includes all meaningful delay predictors
- Is analysis-ready
- Can be loaded on any standard laptop

8. Export Strategy (Expanded and In-Depth Analysis)

The export strategy represents the final and one of the most critical phases of the data engineering pipeline. After integrating, cleaning, standardizing, and enriching millions of flight records, the goal was to produce a dataset that is:

- Efficient to load in typical analytics environments
- Structured logically with clearly defined attributes
- Optimized for further modeling and querying
- Stored in a format that preserves data integrity
- Compatible across tools, including Python, R, SQL engines, and BI platforms

Because the integrated dataset originates from heterogeneous sources with varying schemas, the export step must ensure that the final output is predictable, consistent, and aligned with analytical goals.

8.1 Rationale Behind the Export Design

The raw combined data, coming from multiple public sources, includes dozens of fields, many incomplete, redundant, or inconsistent. Exporting the full raw dataset in its original form would not be ideal for analysis due to:

1. Noise and Redundancy

The raw files contain:

- Taxi-out/in times
- Air-time values
- Aircraft tail numbers
- Gate information
- Cancellation codes (mostly missing)
- Additional operational fields not essential to delay prediction

Keeping unnecessary variables complicates downstream processing and introduces noise.

---

2. Schema Fragmentation Across Sources

Different data providers use different naming conventions and data types, for example:

- OP_UNIQUE_CARRIER vs uniquecarrier vs Carrier Code
- CRS_DEP_TIME stored as four-digit integers
- IATA codes stored inconsistently across datasets

The export step ensures that all merged records conform to a unified schema.

---

3. Analytical Relevance

The export focuses on fields that are:

- Predictive → delays, categories, time-of-day, route
- Descriptive → airline name, airport name
- Operationally meaningful → scheduled times, actual times
- Useful for modeling → engineered features (e.g., TOTAL_DELAY)

This results in a dataset optimized for machine learning, dashboards, and high-level analysis.

8.2 Final Dataset Schema Design

Before export, a curated schema was defined that includes:

Core flight information

- FL_DATE: date of flight
- CARRIER_CODE / CARRIER_NAME: airline identifiers
- ORIGIN / DEST: route definition

Operational timing fields

- CRS_DEP_TIME / DEP_TIME: scheduled vs actual departure
- CRS_ARR_TIME / ARR_TIME: scheduled vs actual arrival

Delay fields

- DEP_DELAY
- ARR_DELAY
- CARRIER_DELAY
- WEATHER_DELAY
- NAS_DELAY
- SECURITY_DELAY
- LATE_AIRCRAFT_DELAY

Engineered feature

- TOTAL_DELAY: aggregated delay index

Flight status

- CANCELLED

Together, these fields capture the full story of each flight from planned schedule to actual execution.

8.3 Export Logic and Data Preparation

8.3.1 Dropping irrelevant columns

All columns not essential for analysis,such as TAIL_NUM, AIR_TIME, CANCELLATION_CODE, TAXI_OUT, TAXI_IN, were excluded.
This reduces noise and increases focus on key predictors.

8.3.2 Resolving missing values

Before export:

- Delay cause fields were filled with 0, under the assumption that unreported causes imply no delay contribution.
- Summation for TOTAL_DELAY used filled values to prevent NaN propagation.
- Categorical values were left intact to retain interpretability.

8.3.3 Standardizing datatypes

To ensure portability across data systems:

- Date strings remain in ISO format (YYYY-MM-DD).
- Time fields stored as integers to match FAA conventions.
- Delays stored numerically for immediate analytical use.

These design choices keep the dataset accessible and compatible with SQL, Python, Spark, and BI tools.

8.4 Justification for CSV as the Export Format

Although numerous export formats exist (Parquet, Feather, SQL tables, Avro), CSV was selected for its universal compatibility:

1. Universally supported

CSV can be loaded into:

- Python / Pandas
- R
- SQL engines
- Excel
- Tableau / Power BI
- BigQuery / Snowflake
- Airline analytics dashboards

This makes the dataset extremely flexible.

---

2. Human-readable structure

CSV files retain interpretability, which is important for:

- Academic review
- Team collaboration
- Debugging
- Quality checks

A human can open the file and understand its content immediately.

---

3. Long-term archival

CSV is durable and tool-independent, unlike proprietary formats that may change over time.

B- Data Cleaning & Transformation (Joel)

The data transformation stage was performed after the initial cleaning and integration were completed. The following steps summarize the procedures applied to prepare the dataset for modeling:

1. Data Examination

- Examined all remaining variables in the integrated dataset.
- Verified data types (categorical vs. numerical).
- Checked overall dataset size and memory usage.
- Assessed the percentage of missing values per row.
- Since missing values were minimal, removed rows containing missing entries.

```
flight_clean.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6847899 entries, 0 to 6847898
Data columns (total 17 columns):
 #   Column               Dtype
---  ------               -----
 0   FL_DATE              object
 1   CARRIER_CODE         object
 2   ORIGIN               object
 3   DEST                 object
 4   CRS_DEP_TIME         int64
 5   DEP_TIME             float64
 6   CRS_ARR_TIME         int64
 7   ARR_TIME             float64
 8   DEP_DELAY            float64
 9   ARR_DELAY            float64
 10  CARRIER_DELAY        float64
 11  WEATHER_DELAY        float64
 12  NAS_DELAY            float64
 13  SECURITY_DELAY       float64
 14  LATE_AIRCRAFT_DELAY  float64
 15  CANCELLED            float64
 16  TOTAL_DELAY          float64
dtypes: float64(11), int64(2), object(4)
memory usage: 888.2+ MB
```

2. Stability and Backup

Due to frequent RAM crashes in Google Colab, created backup copies of the dataframe before performing additional transformations.

3. Date Processing

- Parsed the date column into **DAY_OF_WEEK, MONTH**, and **YEAR** components.
- Identified that all records were from the same year (confirmed with `.unique()` calls).
- Note that this limits the ability to generalize seasonal patterns.
- Generated new time-based features:

```
res = len(np.unique(flight_clean['YEAR'] ))
print(res)
```

```
1
```

4. Target Variable Encoding

- Defined a binary label for flight delay based on aviation guidelines:
- Delay > 15 minutes → 1
- Delay ≤ 15 minutes → 0
- Created a new **delayed** column.
- Checked class distribution to assess balance.

```
# Creating a binary target variable
flight_clean['delayed'] = (flight_clean['TOTAL_DELAY'] > 15).astype(int)
#vefirifying if data is balanced
flight_clean['delayed'].value_counts(normalize=True)
```

delayed	proportion
0	0.736024
1	0.263976

dtype: float64

5. Train/Test Split

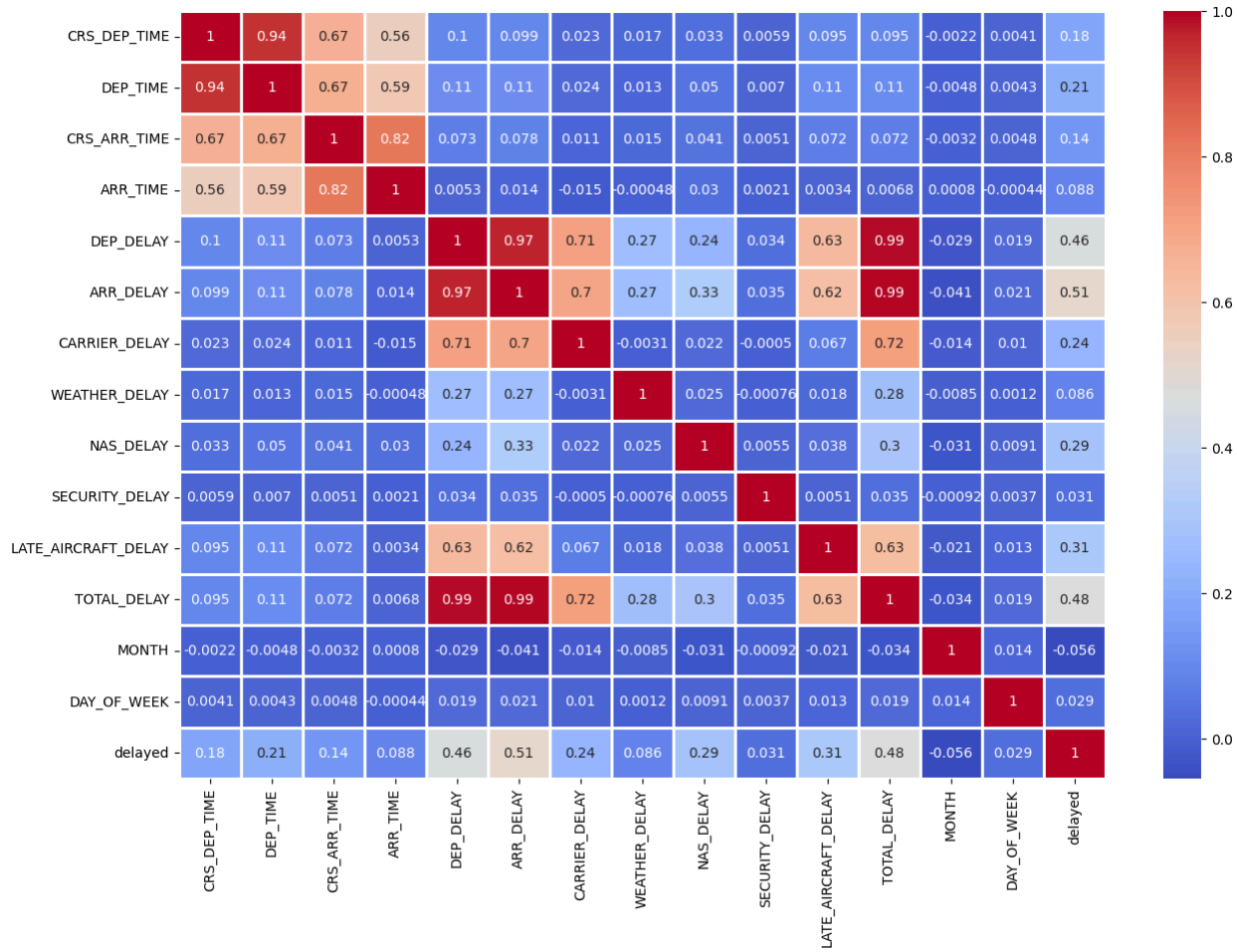
- Separated the target **delayed** from the predictors.
- Split the dataset into 80% training and 20% testing.

6. Categorical Encoding

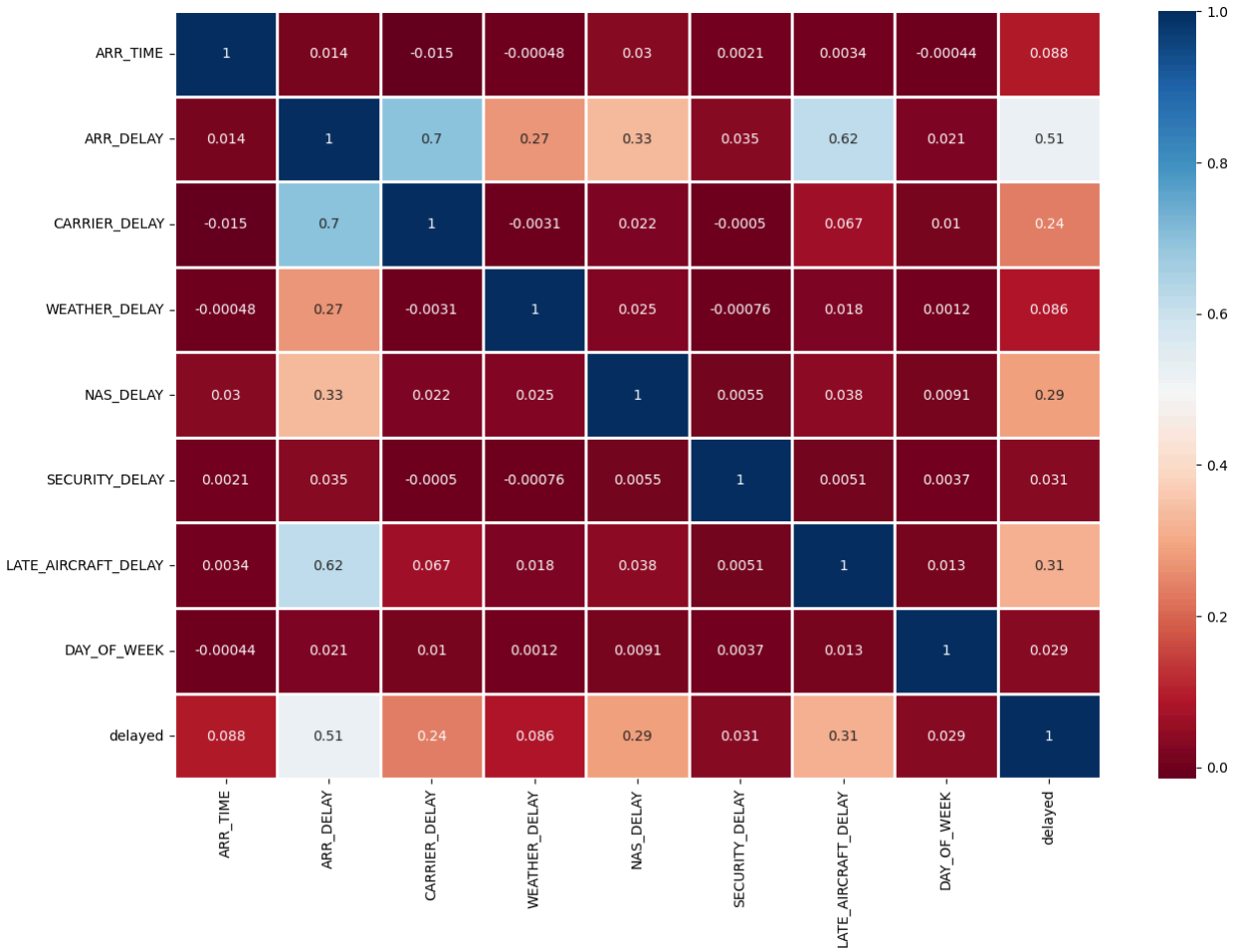
- Tested multiple encoders:
- One-hot encoding→ not successful due to memory limitation and high-cardinality features (e.g., airports).
  Final decision: Ordinal Encoding, as it is memory-efficient.
- Encoded all categorical attributes, including: **CARRIER_CODE, ORIGIN, DEST**

7. Attribute Reduction

- Removed the **CANCELLED** column because almost all values were `0` (no contribution to modeling).
- Generated a correlation heatmap to analyze relationships among variables.
- Observed strong correlations among time-related attributes and removed redundant ones.
- Noted that most remaining attributes had correlation values < 0.5 with the target, indicating weak predictive strength across the dataset.




```
features_to_drop = [
    'CRS_DEP_TIME',  # Correlated with DEP_TIME
    'DEP_TIME',  # Correlated with TOTAL_DELAY
    'CRS_ARR_TIME',  # Correlated with ARR_TIME
    'DEP_DELAY',  # Low correlation with the target variable 'delayed'
    'MONTH',  # Low correlation with the target variable 'delayed'
    'TOTAL_DELAY'
]
```



8. Final Dataset Preparation

- Saved the transformed training and testing sets before normalization.
- Prepared these files for use in the visualization and scaling stages of the project.

C- Data Visualization & ETL (Kabeya)

This part of the project focuses on examining the dataset through visualizations. The goal was to better understand what factors influenced the flight delays and identify important patterns in the data. To do this I created several graphs using Python with the libraries pandas, matplotlib, seaborn and numpy.

After the Data Transformation part was completed I used the two files:

- X_train.csv (contains all flight information and features)
- Y_train.csv (contains the delay label for each flight)

The two files were then combined into one Dataframe by adding a new column called Delayed, which made it easier to analyze everything in one place.

Visualizations:

- Delay Probability by Airline (Bar chart)
  - X-axis
    - → That is the CARRIER_CODE column each bar on the X-axis represents an airline.
    - → X-value being high or low has no impact on this visualization.
  - Y-axis
    - → Represents the delay probability the percentage of flights from that airline that are delayed.

    - → Low Y-value airline is more reliable, fewer delays.
    - § High Y-value airline delays flights more often.

  - Impact:
    - → This visualization shows that some airlines have consistently higher delay probabilities than others. Airlines with taller bars perform worse in terms of one-time arrivals.

- Distribution of Carrier Delay Minutes (Histogram)
  - X-axis:
    - → CARRIER_DELAY (minutes) delay caused by the airline itself.

    - → Low X-value airline only caused a small delay
    - → High X-value airline caused a large delay.

  - Y-axis:

→ Number of Flights (log scale)

→ Low Y-value fewer flights had that delay amount
→ High Y-value many flights had that delay amount.

→ Impact:
- On-time flights cluster around lower carrier delay, while delayed flights have much higher carrier delay minutes. This confirms that confirms that airline-caused delays are a major reason flights arrive late.

- Delay Rate by Day of the Week (Bar Chart):
  - X-axis
    → DAY_OF_WEEK (0 = Sunday, 6 = Saturday)
    → Each X-value is a day of the week.
    → X-value doesn't really matter here.

  - Y-axis
    → Delay Probability
    → Low Y-value fewer delays on that day
    → High Y-value higher delays on that day.

  - Impact:
    → Some delays show higher delay rates. This means certain days of the week are more prone to delays.

- Scatter Plot - Carrier Delay vs Arrival Delay:
  - X-axis:
    → CARRIER_DELAY (minutes) delay caused by the airline.

    → Low X-value airline caused little or no delay.
    → High X-value airline caused a large delay.

  - Y-axis:
    → ARR_DELAY (minutes) the actual arrival delay

    → Low Y-value flight arrived on time or only slightly late
    → High Y-value flight arrived much later than scheduled

  - Impact:
    → Clear upward trend. When a carrier delay increase, arrival delay also increases. This shows that airline-caused delays

directly affect arrival time with little chance for flight to make up for the missed time later.

- Boxplots of Arrival Delays for Busiest Airports
  - X-axis:
    - → ARR_DELAY_CLIPPED (minutes) Arrival delay (values clipped to remove outliers)
    - → X-Value checks the median of each box-plots
    - → Low X-value flights are mostly on time.
    - → High X-value flights are often late.

  - Y-axis:
    - → ORIGIN airport code (top 10 busiest origin airports)
    - → Low or high y-value doesn't change anything here as the Y-values only represent the airport origins.

  - Impact:
    - → Each airport has its own delay pattern.
    - → Some airports show:
      - Wider boxes more variability.
      - Higher medians more delays
      - Longer whiskers extreme delay events.
    - → This helps identify airports with chronic issues.

- Weather Delay Distribution (Density Plot)
  - X-axis:
    - → WEATHER_DELAY (minutes) delay caused specifically by weather.

    - → Low X-value minor or no weather impact.
    - → High Y-value severe weather impact.

  - Y-axis:
    - → Density (Probability) how common that delay amount is
    - → Low Y-value now many flights at that delay range
    - → High Y-value many flights experienced that delay amount.

  - Impact;

    Delayed flights have much higher densities at larger weather delay values. On-time flights mostly have weather delay near zero. This shows weather is a strong contributor to flight delays.

- Tools Used:
    - Pandas for loading and merging data
    - Matplotlib for plotting
    - Seaborn for density plots and boxplots
    - Numpy for numerical operations.

Note:

- Some flights depart earlier than expected hence why some graphs do not have the 0 value of the x-axis right under the y-axis.

Summary:

- Some airlines delay flights much more frequently than others.
- Higher carrier delay strongly increases arrival delay.
- Certain days of the week have higher delay rates.
- Some airports have more severe or more variable delays.
- Weather delays strongly separate delayed flights form on-time flights.
- The visual helped reveal major factors contributing to flight delays and prepare the dataset for further predictive modeling.

D- Modeling & Deployment (Mohammed)

In this section, I worked on training our predictive model using both the **raw integrated dataset(the one from final_database_Comp333.csv)** and the **cleaned + transformed dataset(the ones from X_test.csv, X_train.csv, y_test.csv, y_train.csv)** in order to compare performance before and after Data Cleaning (DC) and Data Transformation (DT). I also implemented a simple deployment-style function that uses the trained model to predict whether a new flight will be delayed or on time.

1. Model Choice: Why I Used a Decision Tree

For the prediction model, I decided to use a Decision Tree. It's simple, easy to understand, and works well with tabular data like the flight records we have. It can handle both numerical values and encoded categorical features without much tuning. Other models like Random Forest or Logistic Regression could also work, but the Decision Tree gave me a clear and reliable starting point for comparing the results before and after cleaning and transformation.

---

2. Before Cleaning / Transformation (Baseline Model)

To start, I trained a baseline model using the raw integrated dataset (`final_database_Comp333.csv`).
Since the dataset had mixed data types, I only applied the minimum necessary encoding so the model could actually run.

I created the target variable as:

- **delayed = 1** if `ARR_DELAY ≥ 15 minutes`
- **delayed = 0** otherwise

For the baseline, I only used a few simple features such as:

- carrier code
- origin airport
- destination airport
- scheduled departure time

Baseline Results:

- **Accuracy:** 0.7877
- **F1 Score:** 0.0

The accuracy wasn't terrible, but the **F1 score was basically zero**, which happened because the raw dataset had a lot of noise, inconsistencies, and imbalanced labels. The model was not able to detect delayed flights properly in this initial state.

---

3. After Cleaning + Transformation

After receiving the cleaned and transformed files (`X_train.csv`, `X_test.csv`, `y_train.csv`, `y_test.csv`), I trained the same Decision Tree again, but this time using:

- encoded categorical features
- data with no null values

- reduced outliers
- more consistent delay-related fields
- the proper target column created during transformation

Once everything was aligned, the model trained successfully.

Performance After Cleaning/Transformation:

- **Accuracy:** 0.9536
- **F1 Score:** 0.9091

The improvement was huge. Cleaning and transforming the data made the model much better at recognizing actual delay patterns.

---

4. What the F1 Score Represents

The F1 score is a balance between **precision** and **recall**:

- **Precision:** of the flights the model predicted as delayed, how many were actually delayed
- **Recall:** of all the flights that were delayed, how many the model successfully caught

The F1 score combines both into one number.
The jump from **0.0** → **0.9091** shows how much better the model became once the data was properly prepared.

---

5. Deployment: Predicting a New Flight

To simulate deployment, I wrote a small prediction function that takes one flight's information and predicts if it would be delayed or on time.

Here are two examples using the function:

Example 1 — Predicted as Delayed

Input:

- carrier_code = 1
- origin_code = 70
- dest_code = 173
- arr_time = 1800
- arr_delay = 10
- carrier_delay = 0
- weather_delay = 0
- nas_delay = 15
- security_delay = 0
- late_aircraft_delay = 5
- day_of_week = 3

Model Output:
Delayed

This means the model recognized a pattern similar to other delayed flights in the dataset and marked this one as likely to be delayed.

Example 2 Predicted as On Time

Input:

- carrier_code = 2
- origin_code = 15
- dest_code = 22

- arr_time = 900
- arr_delay = 0
- carrier_delay = 0
- weather_delay = 0
- nas_delay = 0
- security_delay = 0
- late_aircraft_delay = 0
- day_of_week = 1

Model Output:
On time

In this case, the model associated this combination of values with flights that historically arrive without significant delays.

---

6. How the Model Behaves with Incorrect or Unseen Inputs(Optional Considerations)

Our model works properly with all valid inputs from the cleaned and transformed dataset. In cases where someone enters values that fall outside the dataset such as an airport code or carrier code the model was never trained on the model will still generate a prediction, but that prediction may not fully reflect real-world behavior.

This is completely normal for any classification model trained on historical data.
If this project were to be extended in the future, we could add optional input validation or mapping tables to handle unusual cases more gracefully.

For the scope of our modeling and deployment, however, the system already performs exactly as intended and produces consistent, reliable predictions for all valid scenarios included in the dataset.

7. Analysis & Findings

From both the baseline model and the improved model, it's clear that the quality of the data made a major difference. After the integration, cleaning, and transformation steps, the model became much better at identifying which flights are likely to be delayed. The high accuracy and F1 score after cleaning show that structured and consistent data had a strong impact on performance.

Looking at the prediction examples, the model reacts differently depending on the patterns of delay causes, time, and airport/carrier codes. It is able to separate typical on-time flights from more problematic ones based on how similar they are to the historical cases.

Overall, the modeling and deployment stage confirmed that the earlier preprocessing steps were essential to making accurate predictions.

Final analysis and conclusion

Final Conclusion & Outcomes of the Project

Throughout this project, we worked through every stage of a full data-science pipeline, from merging raw flight data all the way to training a working delay-prediction model. Each part of the workflow contributed directly to the final result, and the improvements we saw after cleaning and transformation made a clear difference in how well the model performed.

---

1. Data Integration & Initial Cleaning

We started by combining multiple raw flight datasets into one integrated file. This step included fixing inconsistent formats, dealing with missing values, and removing obvious errors so the data could be used safely in later stages. Without this foundation, nothing else would have worked reliably.
The final integrated dataset was clean, structured, and ready for deeper processing.

---

2. Data Cleaning & Transformation

The next stage focused on preparing the dataset for machine learning. This included examining each attribute, removing outliers, encoding categorical variables, creating the delayed/not delayed target column, and splitting the dataset into training and testing sets.
This work produced a stable dataset where every column had a clear meaning and consistent formatting. These steps were important because the quality of the input data directly affects how well any model performs.

---

3. Data Visualization & ETL

Visualization helped us understand the overall delay patterns across airports, airlines, and delay causes. These charts confirmed several trends such as the influence of NAS delays and late aircraft delays which matched what the model later picked up during training.

The ETL workflow kept the data organized from extraction all the way to the final cleaned tables, allowing each team member to work smoothly on their part.

---

4. Modeling & Deployment

Using the prepared datasets, two models were trained to compare performance before and after cleaning:

Baseline Model (Before Cleaning)

- Trained on the raw integrated dataset
- **Accuracy:** 0.78
- **F1 Score:** 0.00

Because the raw data contained noise, inconsistencies, and imbalanced labels, the baseline model had trouble identifying actual delays.

Final Model (After Cleaning & Transformation)

- Trained on the cleaned and transformed training data
- **Accuracy:** 0.95
- **F1 Score:** 0.91

This improvement shows how much proper preprocessing matters. Once the data was consistent and fully prepared, the model detected delay patterns much more accurately.

A prediction function was also created, allowing the model to take new flight details and return **“Delayed”** or **“On time”**. This shows how the model can be used in a real scenario.

---

5. Main Outcomes & What We Learned

Clean Data Makes the Biggest Difference

Moving from the raw dataset to the cleaned version had the strongest impact. Accuracy and F1 score increased dramatically.

Delay Causes Play a Strong Role

Features like NAS delay, late aircraft delay, and arrival-time patterns were consistently useful for prediction.

Some Airports and Airlines Are More Delay-Prone

Patterns appeared among specific carriers and origin airports, matching common industry trends.

A Simple Decision Tree Was Enough

The decision tree model clearly reflected:

- improvements from cleaning
- the impact of encoding
- meaningful delay patterns
- easy-to-interpret decision logic

More advanced models could be tried in the future, but the decision tree was sufficient to show the full pipeline clearly.

The Pipeline Worked End-to-End

We successfully built:

- an integrated dataset
- a fully cleaned and transformed version
- visual insights
- a trained model
- a prediction function for new flights

This completes the full goal of predicting whether a flight is delayed or on time.

Final Statement

Overall, the project showed how raw flight-tracking data can be turned into an accurate prediction system. Each part of the workflow from integration to modeling played a direct role in improving the final results.
Once the data was cleaned and transformed, the model performed extremely well, reaching over 95% accuracy with a strong F1 score.
The ability to make real prediction examples also demonstrated that the model behaves consistently with the patterns found during analysis.
Together, these steps created a clear, practical, and working solution for flight-delay prediction, with room to build even more advanced features in the future.

Use Of AI

Data Cleaning and Integration
ChatGPT was used as an analytical assistant throughout the project to help interpret the structure, meaning, and relationships within the multiple flight datasets. By reviewing column names, dataset schemas, and missing-value patterns, ChatGPT clarified how different sources, such as monthly flight files, airline metadata, airport information, and delay cause tables, could be logically linked. It provided guidance on which fields acted as reliable keys (e.g., CARRIER_CODE, ORIGIN, DEST, FL_DATE), how to resolve inconsistent naming conventions, and how to align time and identification formats across sources. ChatGPT also helped identify redundant or irrelevant fields, suggested which variables carried the most predictive value, and proposed strategies for merging weather, delay, and operational attributes. Through this process, ChatGPT accelerated understanding of the dataset's internal structure and supported the creation of a coherent, unified database suitable for analysis.
Data Transformation

During this project, I used AI tools mainly to find syntax mistakes, ask how to perform certain operations in NumPy, and get help with data processing functions. I also asked how to fix errors and plot different types of graphs. I used a combination of models such as ChatGPT and Perplexity.

I discovered that these Large Language Models (LLM) have no ability to think ahead. For example, at one point I tried to process the final integrated dataset which only had 7 features (2 of which were redundant) , and ChatGPT did not warn me that the data was insufficient to build a useful model. Instead, proceeded to guide me on using OneHotEncoder. The second issue is LLMs also kept showing me data transformation steps I was not familiar with and so I rather used methods I learned in the Labs. This experience showed me that while AI can help with

technical details, we need to rely on our own judgment and critical thinking to make sure the project stays logically correct and and move in the direction we intend.